

Autonomous Driving Vehicle

Contents

1	Hardware	2
2	Software	6
2.1	Architecture	6
2.1.1	Embedded Platform	6
2.1.2	Brain	8
2.2	Embedded Platform	9
2.2.1	Peripheral	9
2.2.1.1	I2C (Inter-Integrated Circuit)	9
2.2.1.2	PWM (Pulse Width Modulation)	10
2.2.1.3	UART (Universal Asynchronous Receiver/Transmitter)	12
2.2.1.4	RCC (Reset and Clock Control)	13
2.2.1.5	Timer	14
2.2.2	Sensor	14
2.2.2.1	IMU: BNO055	14
2.2.2.2	Encoder: AS5600	16
2.2.3	Actuator	17
2.2.3.1	Steering: 180° PWM Servo	17
2.2.3.2	Propulsion: Brushless Motor & ESC	18
2.3	Brain	18
2.3.1	Module Communication	18
2.3.1.1	Kernel	18
2.3.1.2	Inter Process Communication	18
2.3.1.3	Pub/Sub Model	19
2.3.1.4	Latest-wins delivery	20
2.3.1.5	C API / C++ Binding	20
2.3.1.6	Python Binding	21
2.3.1.7	Node/Topic	21
2.3.2	IO	22
2.3.3	Perception	22
2.3.3.1	Camera	22
2.3.3.2	Lane Detection	23
2.3.3.3	Object Detection	24
2.3.3.4	Ego State	24
2.3.4	Localization	25
2.3.5	Planning	25
2.3.6	Control	25
2.4	Monitor	25
2.5	Simulation	25
3	Source & Description	25

1 Hardware

Vehicle Platform

STM32F401 NUCLEO

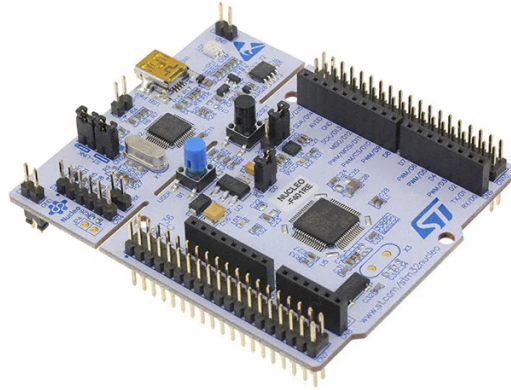


Figure 1: STM32F401 NUCLEO board

Property	Value
MCU	STM32F401RE (ARM Cortex-M4)
Clock	Up to 84 MHz
Flash	512 KB
RAM	96 KB
Interface	UART, I2C, SPI, GPIO, ADC, PWM
Role	Low-level control: motor, servo, sensor reading

Table 1: STM32F401 NUCLEO specifications

Raspberry Pi 5

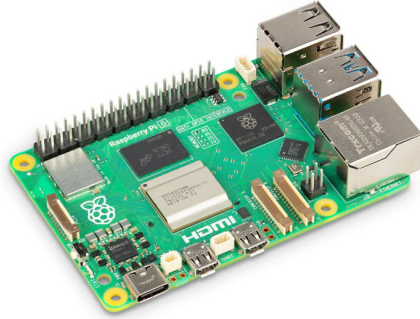


Figure 2: Raspberry Pi 5

Property	Value
CPU	Broadcom BCM2712, quad-core Cortex-A76 @ 2.4 GHz
RAM	4 GB / 8 GB LPDDR4X
OS	Ubuntu 24.04 (64-bit)
Interface	UART, USB, CSI camera, Ethernet, Wi-Fi
Role	High-level brain: perception, planning, control

Table 2: Raspberry Pi 5 specifications

Encoder: AS5600

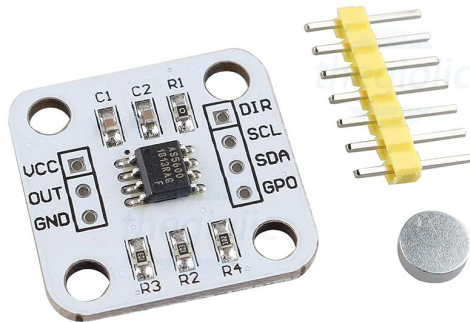


Figure 3: AS5600 magnetic rotary encoder module

Property	Value
Type	Contactless magnetic rotary encoder
Resolution	12-bit (4096 positions/revolution)
Interface	I2C (address 0x36)
Supply voltage	3.3 V – 5 V
Output	Absolute angle 0°–360°
Role	Measures wheel rotation for speed estimation

Table 3: AS5600 encoder specifications

IMU: BNO055

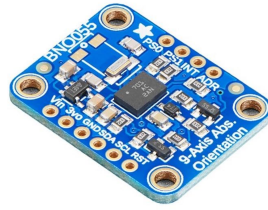


Figure 4: BNO055 9-axis IMU module

Property	Value
Axes	9-axis (accelerometer + gyroscope + magnetometer)
Interface	I2C (address 0x28 / 0x29)
Output	Euler angles: yaw, pitch, roll (degrees)
Supply voltage	3.3 V
Fusion mode	NDOF (on-chip sensor fusion)
Role	Provides heading angle (yaw) for state estimation

Table 4: BNO055 IMU specifications

Actuator: Steering Servo



Figure 5: 180° steering servo

Property	Value
Type	PWM position servo
Range	0°–180°
Control	PWM 50 Hz, pulse width 1–2 ms
Supply voltage	5 V
Role	Controls front wheel steering angle

Table 5: Steering servo specifications

Simulator

- Encoder: AS5600
- Motor
- 3D Model design

2 Software

2.1 Architecture

2.1.1 Embedded Platform

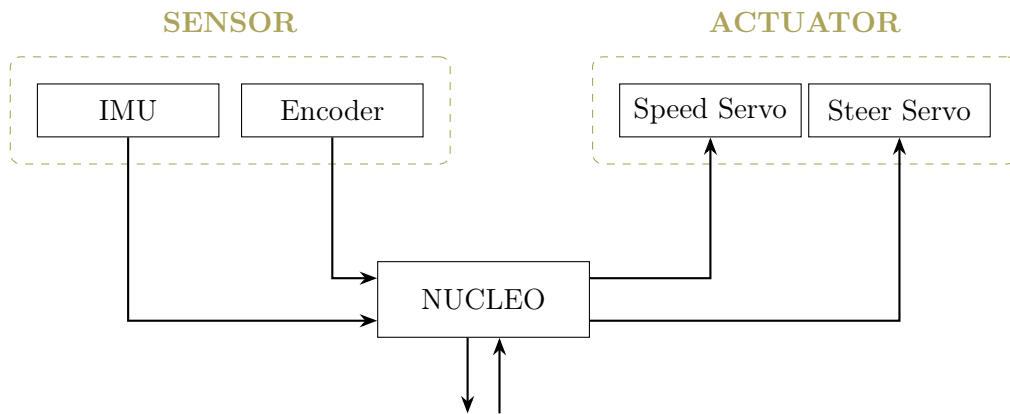


Figure 6: Embedded Platform: NUCLEO reads sensors via I2C, drives actuators via PWM, communicates with Raspberry Pi 5 via UART

2.1.2 Brain

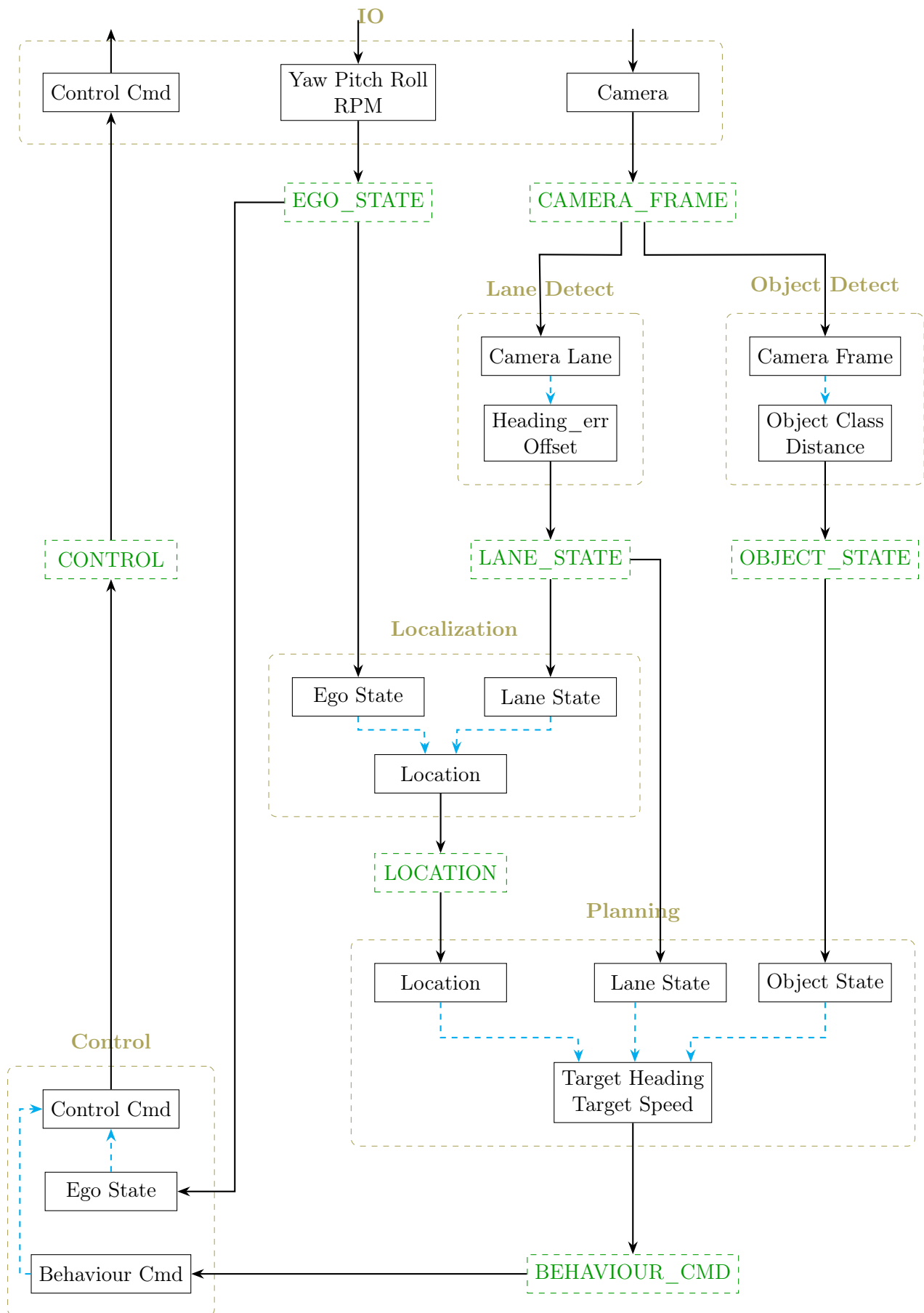


Figure 7: Brain architecture: IO layer

2.2 Embedded Platform

2.2.1 Peripheral

2.2.1.1 I2C (Inter-Integrated Circuit) I2C is a two-wire synchronous serial protocol using a shared clock line (SCL) and a bidirectional data line (SDA). Communication is initiated by a START condition (SDA falls while SCL is high), followed by a 7-bit device address, a read/write bit, an ACK from the slave, data bytes, and a STOP condition (SDA rises while SCL is high). In this project, I2C1 (PB8=SCL, PB9=SDA) communicates with BNO055 (0x28) and AS5600 (0x36) at 100 kHz.

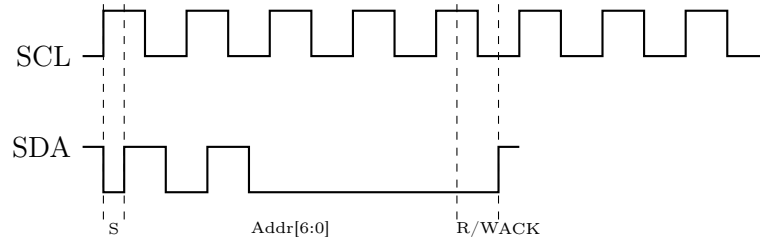


Figure 8: I2C transaction: START, 7-bit address (0x28), R/W bit, ACK

Function	Signature	Description
I2C1_Master_Init	(void)	Configure I2C1 at 100 kHz on PB8/PB9
I2C1_WriteReg	(dev, reg, data)	Write one byte to a device register
I2C1_ReadMulti	(dev, reg, buf, len)	Read <code>len</code> bytes starting from <code>reg</code>

Table 6: I2C driver functions

Function: I2C1_Master_Init()

Configures GPIO PB8/PB9 as AF4 (open-drain, pull-up), enables clocks, then sets I2C1 timing for 100 kHz with PCLK1 = 16 MHz.

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOBEN	= (1<<1)	Enable GPIOB clock
RCC->APB1ENR	I2C1EN	= (1<<21)	Enable I2C1 clock
GPIOB->MODER	PB8,PB9	0b10 each	Alternate function mode
GPIOB->OTYPER	PB8,PB9	1 each	Open-drain output
GPIOB->OSPEEDR	PB8,PB9	0b11 each	High speed
GPIOB->PUPDR	PB8,PB9	0b01 each	Pull-up
GPIOB->AFR[1]	PB8,PB9	0x4 each	AF4 = I2C1
I2C1->CR1	PE	0 (clear)	Disable I2C before config
I2C1->CR2	FREQ	16	PCLK1 = 16 MHz
I2C1->CCR	CCR	80	100 kHz: $T_{high} = T_{low} = \frac{80}{16\text{MHz}} = 5 \mu s$
I2C1->TRISE	TRISE	17	Max rise time: $\frac{1000 ns}{62.5 ns} + 1 = 17$
I2C1->CR1	ACK	= (1<<10)	Enable auto-ACK
I2C1->CR1	PE	= (1<<0)	Enable I2C peripheral

Table 7: I2C1_Master_Init() register configuration

Function: I2C1_WriteReg(dev, reg, data)

Sends START → device address (write) → register address → data byte → STOP.

Register	Action	Value / Condition	Purpose
I2C1->CR1	Write	= START (1<<8)	Generate START condition
I2C1->SR1	Poll	wait SB==1	START sent
I2C1->DR	Write	dev<<1	Send address + Write bit
I2C1->SR1/SR2	Read	wait ADDR==1, clear by reading SR2	Address ACKed
I2C1->SR1	Poll	wait TXE==1	Tx buffer empty
I2C1->DR	Write	reg	Send register address
I2C1->SR1	Poll	wait TXE==1	Tx buffer empty
I2C1->DR	Write	data	Send data byte
I2C1->SR1	Poll	wait BTF==1	Byte transfer finished
I2C1->CR1	Write	= STOP (1<<9)	Generate STOP condition

Table 8: I2C1_WriteReg() register sequence

Function: I2C1_ReadMulti(dev, reg, buf, len)

Sends register address (write phase), then Repeated START, reads `len` bytes (NACK + STOP on last byte).

Register	Action	Value / Condition	Purpose
I2C1->CR1	Write	= START	START condition
I2C1->DR	Write	dev<<1	Address + Write
I2C1->DR	Write	reg	Register pointer
I2C1->SR1	Poll	wait BTF==1	Write phase done
I2C1->CR1	Write	= START	Repeated START
I2C1->DR	Write	(dev<<1) 1	Address + Read bit
I2C1->CR1	Write (last byte)	&= ~ACK	Disable ACK before last byte
I2C1->CR1	Write (last byte)	= STOP	STOP after last byte
I2C1->SR1	Poll each byte	wait RXNE==1	Data register not empty
I2C1->DR	Read	buf[i]	Read received byte

Table 9: I2C1_ReadMulti() register sequence

2.2.1.2 PWM (Pulse Width Modulation) PWM generates a periodic digital signal with fixed frequency and variable duty cycle. The STM32 timer compares the counter value against a capture/compare register (CCR); output is high when `CNT < CCR` and low otherwise. PWM is used to control the servo steering angle and to send throttle commands to the ESC. Typical servo period is 20 ms (50 Hz); pulse width 1–2 ms maps to 0°–180°.

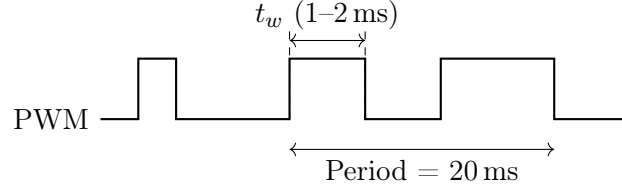


Figure 9: PWM signal: varying pulse width, fixed 50 Hz period

Function	Signature	Description
PWM_Init	(void)	Configure TIM2 CH1/CH2 at 50 Hz on PA0/PA1
Servo_Speed_Set	(us)	Set ESC pulse width directly in μ s (1000–2000)
Servo_Steering_Set	(us)	Set steering pulse width directly in μ s (1100–1900)
Servo_Speed_Set_RPM	(rpm)	Convert RPM $[-500, +500]$ to CCR1 and write
Servo_Steering_Set_Deg	(deg)	Convert angle $[-40, +40]$ to CCR2 and write

Table 10: PWM driver functions

Function: PWM_Init()

Configures PA0 (TIM2_CH1, speed servo) and PA1 (TIM2_CH2, steering servo) as AF1 PWM outputs at 50 Hz with PCLK1 = 16 MHz.

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOAEN	$ = (1 \ll 0)$	Enable GPIOA clock
RCC->APB1ENR	TIM2EN	$ = (1 \ll 0)$	Enable TIM2 clock
GPIOA->MODER	PA0, PA1	0b10 each	Alternate function mode
GPIOA->AFR[0]	PA0, PA1	0x1 each	AF1 = TIM2_CH1/CH2
TIM2->PSC	PSC	15 (16-1)	Tick = 16 MHz/16 = 1 MHz
TIM2->ARR	ARR	19999 (20000-1)	Period = 20 ms $\rightarrow$ 50 Hz
TIM2->CCR1	CCR1	1500	Speed servo: neutral (1.5 ms)
TIM2->CCR2	CCR2	1500	Steering servo: center (1.5 ms)
TIM2->CCMR1	OC1M	0b110 (bits [6:4])	PWM mode 1 on CH1
TIM2->CCMR1	OC1PE	1 (bit 3)	Preload enable CH1
TIM2->CCMR1	OC2M	0b110 (bits [14:12])	PWM mode 1 on CH2
TIM2->CCMR1	OC2PE	1 (bit 11)	Preload enable CH2
TIM2->CCER	CC1E	1 (bit 0)	Enable CH1 output
TIM2->CCER	CC2E	1 (bit 4)	Enable CH2 output
TIM2->EGR	UG	1 (bit 0)	Force reload PSC/ARR
TIM2->CR1	CEN	$ = (1 \ll 0)$	Start counter

Table 11: PWM_Init() register configuration

Functions: Servo_Speed_Set(us), Servo_Steering_Set(us)

Function	Register	Value	Range
Servo_Speed_Set(us)	TIM2->CCR1	us	1000–2000 μ s
Servo_Steering_Set(us)	TIM2->CCR2	us	1100–1900 μ s
Servo_Speed_Set_RPM(rpm)	TIM2->CCR1	1500 - (int)rpm	rpm: -500..+500
Servo_Steering_Set_Deg(deg)	TIM2->CCR2	1500 + (int)(deg*10)	deg: -40..+40°

Table 12: Servo control functions and their CCR values

2.2.1.3 UART (Universal Asynchronous Receiver/Transmitter) UART is an asynchronous serial protocol with no shared clock. A frame consists of: 1 start bit (low), 8 data bits (LSB first), optional parity, and 1–2 stop bits (high). Both ends must agree on the same baud rate. This project uses 115200 baud, 8N1 (no parity, 1 stop bit). The STM32 transmits sensor data to the Raspberry Pi 5 over UART in ASCII frames: @IMU:yaw,pitch,roll;;\r\n and @RPM:rpm;;\r\n.

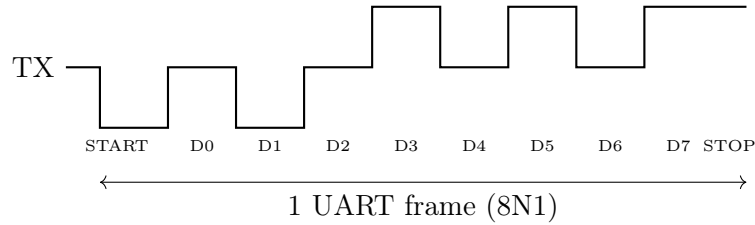


Figure 10: UART frame: start bit, 8 data bits (LSB first), stop bit

Function	Signature	Description
UART1_init	(void)	Configure USART1 (PA9/PA10) at 115200 baud 8N1
UART2_init	(void)	Configure USART2 (PA2/PA3) with RXNE interrupt
UART1_Transmit	(data)	Send one byte via USART1 TX
UART2_Transmit	(data)	Send one byte via USART2 TX

Table 13: UART driver functions

Function: `UART1_init()` — PA9 TX, PA10 RX, AF7, 115200 baud 8N1, PCLK2 = 16 MHz.

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOAEN	= (1<<0)	Enable GPIOA clock
RCC->APB2ENR	USART1EN	= (1<<4)	Enable USART1 clock
GPIOA->MODER	PA9, PA10	0b10 each	Alternate function mode
GPIOA->AFR[1]	PA9, PA10	0x7 each	AF7 = USART1
USART1->BRR	DIV_Mantissa	8<<4	115200 baud: USARTDIV = 8.6875
	DIV_Fraction	11	
USART1->CR1	OVER8	0 (bit 15)	Oversampling by 16
USART1->CR1	M	0 (bit 12)	8-bit word length
USART1->CR1	PCE	0 (bit 10)	No parity
USART1->CR1	RE	1 (bit 2)	Receiver enable
USART1->CR1	TE	1 (bit 3)	Transmitter enable
USART1->CR1	UE	1 (bit 13)	USART enable

Table 14: UART1_init() register configuration

Function: UART2_init() — PA2 TX, PA3 RX, AF7, 115200 baud, with RXNE interrupt (NVIC IRQ 38).

Register	Field	Value	Effect
RCC->APB1ENR	USART2EN	= (1<<17)	Enable USART2 clock
GPIOA->MODER	PA2, PA3	0b10 each	Alternate function mode
GPIOA->AFR[0]	PA2, PA3	0x7 each	AF7 = USART2
USART2->BRR		(8<<4) 11	115200 baud @16 MHz
USART2->CR1	RXNEIE	1 (bit 5)	Enable RXNE interrupt
USART2->CR1	RE, TE, UE	1 each	Enable RX, TX, USART
NVIC->ISER[1]	IRQ38	1<<(38-32)	Enable USART2 IRQ in NVIC

Table 15: UART2_init() register configuration

Functions: UART1_Transmit(data), UART2_Transmit(data)

Register	Action	Condition / Value	Purpose
USARTx->SR	Poll	wait TXE==1 (bit 7)	TX buffer empty
USARTx->DR	Write	data	Load byte to transmit
USARTx->SR	Poll	wait TC==1 (bit 6)	Transmission complete

Table 16: UART transmit sequence

ISR: USART2_IRQHandler() — Triggered on each received byte, reads USART2->DR to clear RXNE, passes byte to UART_Parse_Byte() which accumulates into a command buffer and calls UART_Parse_Command() on ;; delimiter.

2.2.1.4 RCC (Reset and Clock Control) The RCC peripheral on the STM32F401 manages system clock configuration and peripheral clock gating. The internal HSI oscillator (16 MHz) or external HSE crystal can be fed into the PLL to generate the system clock (SYSCLK). On the STM32F401 NUCLEO, SYSCLK is configured to 84 MHz. Peripheral buses APB1 (max 42 MHz) and APB2 (max 84 MHz) are derived via prescalers. Each peripheral must have its bus clock explicitly enabled via RCC_APBxENR before use.

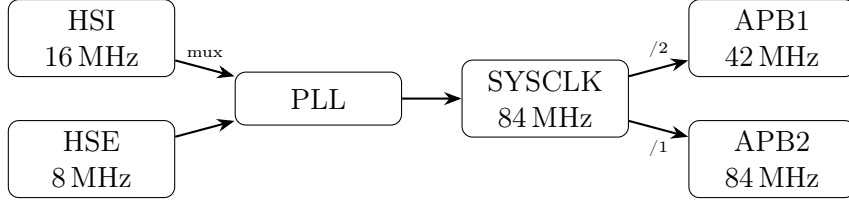


Figure 11: STM32F401 clock tree (simplified): HSI/HSE $\rightarrow$ PLL $\rightarrow$ SYSCLK $\rightarrow$ APB buses

2.2.1.5 Timer STM32 general-purpose timers (TIM2–TIM5) use an up-counter that increments each tick. The tick frequency is $\frac{f_{APB} \times 2}{PSC+1}$; the counter resets at ARR (Auto-Reload Register), producing an overflow period of $\frac{ARR+1}{f_{tick}}$. In this project:

- **TIM1/TIM4** — PWM output for servo and ESC ($f = 50$ Hz, $ARR = 19999$, $PSC = 83$)
- **TIM2** — encoder interface mode, counts quadrature edges from AS5600 (via GPIO simulation)
- **TIM3** — periodic interrupt for UART transmit scheduling (1 kHz)

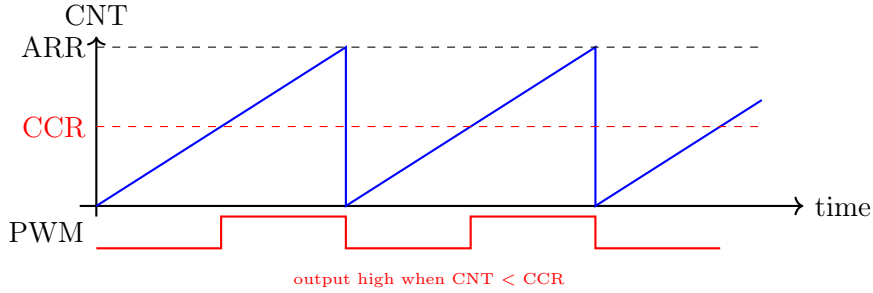


Figure 12: Timer up-count mode: counter ramps to ARR, resets; PWM output high while $CNT < CCR$

2.2.2 Sensor

2.2.2.1 IMU: BNO055 Communication: I2C, address 0x29 (ADDR pin high). The MCU is master; BNO055 is slave. Bus speed 100 kHz matching `I2C1_Master_Init()`.

Function	Signature	Description
BNO055_Init	(void)	Set NDOF fusion mode, configure axes
BNO055_ReadEuler_raw	(x, y, z)	Read raw Euler angles (yaw/pitch/roll)
BNO055_ReadCalibStatus	(void)	Read CALIB_STAT register (0x00–0xFF)
IMU_HealthCheck	(ex, ey, ez)	Detect stuck-zero fault; stop watchdog if faulty

Table 17: BNO055 driver functions

Key registers used in implementation:

Macro	Address	Description
BN0055_CHIP_ID	0x00	Chip ID — expected value 0xA0
BN0055_PAGE_ID	0x07	Page select — write 0x00 to use page 0
BN0055_OPR_MODE	0x3D	Operation mode: 0x00=CONFIG, 0x0C=NDOF
BN0055_PWR_MODE	0x3E	Power mode: 0x00=Normal, 0x01=LowPower, 0x02=Suspend
BN0055_UNIT_SEL	0x3B	Unit select: bit0=0 → degrees output
BN0055_EUL_X_LSB	0x1A	Euler heading (yaw) LSB — 1 LSB = 1/16°
BN0055_EUL_X_MSB	0x1B	Euler heading (yaw) MSB
BN0055_EUL_Y_LSB	0x1C	Euler pitch LSB
BN0055_EUL_Y_MSB	0x1D	Euler pitch MSB
BN0055_EUL_Z_LSB	0x1E	Euler roll LSB
BN0055_EUL_Z_MSB	0x1F	Euler roll MSB
BN0055_CALIB_STAT	0x35	Calibration status — bits [7:6]=sys, [5:4]=gyro, [3:2]=accel, [1:0]=r

Table 18: BN0055 registers used in implementation

Function: BN0055_Init()

Step	I2C Write reg	Value	Purpose
1	OPR_MODE (0x3D)	0x00	Enter CONFIG mode before changing settings
2	PAGE_ID (0x07)	0x00	Select page 0 (sensor/output registers)
3	PWR_MODE (0x3E)	0x00	Set Normal power mode
4	OPR_MODE (0x3D)	0x0C	Enter NDOF fusion mode (9-DOF)

Table 19: BN0055_Init() I2C write sequence

Function: BN0055_ReadEuler_raw(x, y, z)

Burst-reads 6 bytes starting from EUL_X_LSB (0x1A). Each axis is a signed 16-bit little-endian value; 1 LSB = 1/16°.

I2C operation	Reg	Bytes	Decode
Read 6 bytes from	0x1A	b[0..5]	
Heading (yaw) X	0x1A-0x1B	b[1]«8 b[0]	int16_t, divide by 16 → degrees
Pitch Y	0x1C-0x1D	b[3]«8 b[2]	int16_t, divide by 16 → degrees
Roll Z	0x1E-0x1F	b[5]«8 b[4]	int16_t, divide by 16 → degrees

Table 20: BN0055_ReadEuler_raw() data decode

Function: BN0055_ReadCalibStatus()

Reads 1 byte from CALIB_STAT (0x35). Each sensor pair of bits = 3 means fully calibrated.

Bits	Sensor	Value 3 = fully calibrated
[7:6]	System	Overall fusion calibration
[5:4]	Gyro	Gyroscope
[3:2]	Accel	Accelerometer
[1:0]	Mag	Magnetometer

Table 21: CALIB_STAT bit fields

Function: IMU_HealthCheck(ex, ey, ez)

Called after each BN0055_ReadEuler_raw(). If all three axes are zero for 5 consecutive reads, the IMU is flagged as faulty and the watchdog (Feed_WD()) stops being fed — triggering a system reset.

2.2.2.2 Encoder: AS5600 Communication: I2C, fixed address 0x36. No init sequence required — the device is ready immediately after power-on. Only reading is performed at runtime.

Function	Signature	Description
AS5600_ReadStatus	(void)	Read STATUS register (magnet detection flags)
AS5600_ReadAngleRaw	(void)	Read 12-bit raw angle (0–4095)
AS5600_ReadAngleDeg	(void)	Convert raw angle to degrees (0–360°)
AS5600_ReadAngularSpeedDeg	(dt_sec)	Compute angular velocity (°/s) via finite difference

Table 22: AS5600 encoder driver functions

Key registers used in implementation:

Macro	Address	Description
AS5600_STATUS	0x0B	Magnet status: bit5=MH (too strong), bit4=ML (too weak), bit3=MD (magnet detected)
AS5600_ANGLE_MSB	0x0E	Processed angle high byte [11:8]
AS5600_ANGLE_LSB	0x0F	Processed angle low byte [7:0]

Table 23: AS5600 registers used in implementation

Function: AS5600_ReadStatus()

I2C operation	Reg	Bytes	Check
Read 1 byte	0x0B	v	bit3 (MD) must be 1 for valid readings

Table 24: AS5600_ReadStatus() operation

Function: AS5600_ReadAngleRaw()

I2C operation	Reg	Bytes	Decode
Read 2 bytes	0x0E	buf[0..1]	raw = (buf[0] << 8 buf[1]) & 0xFFFF

Table 25: AS5600_ReadAngleRaw() — returns 12-bit value 0–4095

Function: `AS5600_ReadAngleDeg()`

Calls `AS5600_ReadAngleRaw()`, converts with:

$$\text{degrees} = \frac{\text{raw} \times 360}{4096}$$

Function: `AS5600_ReadAngularSpeedDeg(dt_sec)`

Computes angular velocity in $^{\circ}/\text{s}$ from two consecutive raw readings. Handles $0^{\circ}/360^{\circ}$ wrap-around by treating a $\text{delta} > 2048$ or < -2048 as a wrap.

Step	Description
1	Read <code>curr_raw</code> via <code>AS5600_ReadAngleRaw()</code>
2	<code>delta = (int16_t)(curr_raw - prev_raw)</code>
3	If <code>delta > 2048</code> : <code>delta -= 4096</code> (wrapped CW)
4	If <code>delta < -2048</code> : <code>delta += 4096</code> (wrapped CCW)
5	<code>delta_deg = delta * 360.0f / 4096.0f</code>
6	Return <code>delta_deg / dt_sec</code> ($^{\circ}/\text{s}$)

Table 26: `AS5600_ReadAngularSpeedDeg()` algorithm

2.2.3 Actuator

The vehicle uses two actuators driven by PWM signals from the STM32 at 50 Hz: a position servo for steering and a brushless motor with ESC for propulsion. Both are commanded via TIM2 (PA0/PA1, AF1), configured with $PSC = 15$, $ARR = 19999$ so that CCR units map directly to microseconds (1 tick = $1 \mu\text{s}$).

2.2.3.1 Steering: 180° PWM Servo The front axle is steered by a standard RC position servo. The STM32 writes the desired pulse width to TIM2->CCR2 (PA1, TIM2_CH2). Center position corresponds to $1500 \mu\text{s}$; each degree of steering offset maps to $10 \mu\text{s}$, giving an effective range of ± 40 (1100 – $1900 \mu\text{s}$).

Property	Value
Type	PWM position servo (180°)
Control signal	PWM 50 Hz, pulse width 1.1–1.9 ms
Center (straight)	$1500 \mu\text{s}$
Angular mapping	$CCR2 = 1500 + \lfloor \theta_{deg} \times 10 \rfloor$, $\theta \in [-40, +40]$
Supply voltage	5 V
STM32 pin	PA1 — TIM2_CH2

Table 27: Steering servo specifications and CCR mapping

The Brain publishes a `ControlCmd` message with field `steer_deg` $\in [-30, +30]$ (positive = right, negative = left). The STM32 receives this value over UART and calls `Servo_Steering_Set_Deg(deg)`, which writes $1500 + \lfloor deg \times 10 \rfloor$ to TIM2->CCR2.

2.2.3.2 Propulsion: Brushless Motor & ESC The rear axle is driven by a brushless DC (BLDC) motor connected through a 7:1 gear reduction to the rear wheels (wheel radius 6.5 cm). Speed is commanded via an Electronic Speed Controller (ESC) that accepts the same 50 Hz PWM protocol as a servo: 1500 μ s = neutral/brake, values below 1500 reverse, values above 1500 drive forward.

Property	Value
Motor type	Brushless DC (BLDC)
Speed controller	ESC, PWM 50 Hz
Neutral / brake	1500 μ s
Forward range	1500–2000 μ s
Reverse range	1500–1000 μ s
RPM mapping	$CCR1 = 1500 - \lfloor rpm \rfloor$, $rpm \in [-500, +500]$
Gear ratio	7:1 (motor to wheel)
Wheel radius	6.5 cm
Speed equation	$v \text{ [cm/s]} = \frac{rpm}{7} \times 2\pi \times 6.5/60$
STM32 pin	PA0 — TIM2_CH1

Table 28: Brushless motor and ESC specifications

The Brain publishes `ControlCmd.rpm` $\in [-500, +500]$ (positive = forward). The STM32 calls `Servo_Speed_Set_RPM(rpm)`, writing $1500 - \lfloor rpm \rfloor$ to `TIM2->CCR1`. Motor RPM feedback is read back via the AS5600 encoder on the rear axle and reported to the Brain in the `@RPM:value;;\r\n` UART frame, where actual wheel speed is recovered as:

$$v = \frac{rpm_{\text{motor}}}{7} \times 2\pi \times 0.065 \text{ [m/s]}$$

2.3 Brain

2.3.1 Module Communication

2.3.1.1 Kernel ...

2.3.1.2 Inter Process Communication Inter-Process Communication (IPC) refers to mechanisms that allow independent OS processes to exchange data. On Linux, the kernel provides several IPC primitives, each with different trade-offs in latency, throughput, ordering guarantees, and persistence.

Mechanism	Characteristics	Use case	Used in project
POSIX Message Queue	Kernel-buffered, ordered FIFO queue; fixed max depth (<code>msg_max</code>); supports blocking/non-blocking send/recv	Low-rate, queued messages where every message must be delivered	Yes — MQ transport
Shared Memory (SHM)	Fastest IPC: processes map the same physical page; no kernel copy; requires explicit synchronization	High-rate data where only the latest value matters (sensor frames, state)	Yes — SHM latest-wins
Unix Domain Socket	Stream or datagram socket over a file path; reliable, ordered; higher latency than SHM	Bidirectional communication between local processes	Not used
TCP Socket	Network socket; works across machines; adds TCP overhead	Remote monitoring, cross-host communication	Not used

Table 29: Linux IPC mechanism comparison

This project implements two transports inside `libipc`:

- **POSIX MQ** (`mq_transport.c`): wraps `mq_open/mq_send/mq_receive`. Queue depth is clamped to the system limit (`/proc/sys/fs/mqueue/msg_max`, default 10). Used for topics where every message must be consumed (e.g. `CONTROL_CMD`).
- **Shared Memory** (`shm.c`): publisher creates a named SHM object via `shm_open` + `ftruncate`, writes a fixed-size slot protected by a `pthread_rwlock`. Subscriber opens (does *not* create) the same object and reads the latest value. Used for high-rate sensor data where stale frames are discarded (e.g. `CAMERA_FRAME`, `EGO_STATE`).

2.3.1.3 Pub/Sub Model The project adopts a publish/subscribe pattern on top of the two transports. Each logical data stream is identified by a *topic*; nodes are decoupled — a publisher does not know who is listening, and a subscriber does not know the publisher’s identity.

- **Node** — an independent OS process that owns one or more publishers or subscribers. Each node links against `liblibipc` and calls the C API directly.
- **Topic** — a named channel identified by a `TopicId` enum value (e.g. `CAMERA_FRAME=1`, `EGO_STATE=2`, `LANE_STATE=3`, `CONTROL_CMD=4`). Each topic has a fixed payload type, transport kind (MQ or SHM), and drop policy, declared in `topic.c`.

- **Publish** — a node calls `ipc_publish()` with a topic ID and a pointer to a POD struct. The bus layer routes the write to the correct transport. The publisher must start before any subscriber.
- **Subscribe** — a node calls `ipc_subscribe()` then `ipc_recv()` (blocking or timed). For SHM topics the subscriber receives the latest written value; for MQ topics it dequeues in FIFO order.

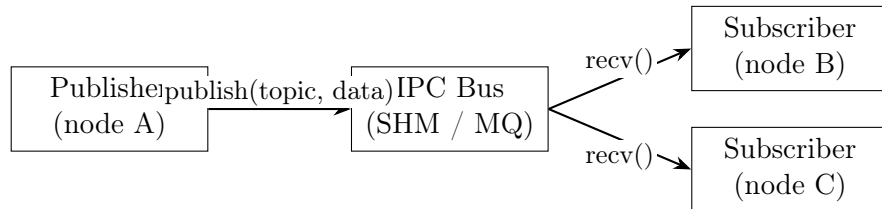


Figure 13: Pub/Sub model: publisher writes to the bus; multiple subscribers receive independently

2.3.1.4 Latest-wins delivery Both transports follow a single *latest-wins* rule, chosen for a real-time control loop where the freshest value matters and a producer must never block:

- **SHM** — the publisher overwrites the slot; a subscriber always reads the most recent value and may skip intermediate samples.
- **MQ** — if the queue is full on send, the oldest message is dropped to make room and the new one is enqueued at the tail. The send never blocks.

There is no configurable drop policy: every topic behaves the same way, so a slow consumer loses old data rather than stalling the producer.

2.3.1.5 C API / C++ Binding The public C API (`libipc/include/ipc/bus.h`) exposes six functions, split into a publisher side and a subscriber side. All names follow a common `publish/subscribe/push/pull` convention shared by the two transport layers underneath.

Role	Function	Description
Publisher	<code>ipc_publish(id, size)</code>	Create the SHM/MQ object, return a handle
	<code>ipc_push(pub, msg, size)</code>	Send one message
	<code>ipc_unpublish(pub)</code>	Detach and unlink the object, free handle
Subscriber	<code>ipc_subscribe(id)</code>	Attach to the existing object, return a handle
	<code>ipc_pull(sub, buf, size, ms)</code>	Receive next message (0=ok, 1=timeout, -1=err)
	<code>ipc_unsubscribe(sub)</code>	Detach and free handle

Table 30: libipc public C API (`ipc/bus.h`)

Internally the API dispatches to one of two transport modules, which share the same naming scheme. The descriptor table (`topic.c`) decides which one a topic uses.

Both modules expose the same set of operations; the table lists the MQ names (the SHM module mirrors them with an `shm_` prefix).

MQ (mq.c)	SHM (shm.c)	Role
mq_publish	shm_publish	create + open object (publisher)
mq_subscribe	shm_subscribe	attach to object (subscriber)
mq_push	shm_push	send one message
mq_pull	shm_pull	receive one message
mq_disconnect	shm_disconnect	close handle
mq_remove	shm_remove	unlink object (publisher)

Table 31: Transport-layer modules (ipc/mq.h, ipc/shm.h)

The C++ binding (`libipc/bindings/cpp/include/ipc/bus.hpp`) wraps the C API into RAII templates: `ipc::Publisher<T>` and `ipc::Subscriber<T>`, where `T` is the message struct type. The constructor opens the endpoint and the destructor closes it automatically; `send()` maps to `ipc_push` and `poll()` to `ipc_pull`.

2.3.1.6 Python Binding The Python binding (`libipc/bindings/python/ipc.py`) uses `ctypes` to call the shared library `liblibipc.so` at runtime. Message structs are defined in `ipc_schema.py` as `ctypes.Structure` subclasses mirroring the C headers in `messages/`. Usage pattern:

1. `import ipc, ipc_schema` (after inserting the bindings path into `sys.path`).
2. `pub = ipc.Publisher(ipc_schema.TopicId.LANE_STATE, size)` — open publisher.
3. Fill a `ipc_schema.LaneState()` struct and call `pub.publish(msg)`.
4. `sub = ipc.Subscriber(ipc_schema.TopicId.EGO_STATE, ipc_schema.EgoState);`
`msg = sub.poll(timeout_ms=100).`

2.3.1.7 Node/Topic

Publisher node	Topic	Subscriber node
serial	EGO_STATE	localization, control
camera	CAMERA_FRAME	lane, object_detection
lane	LANE_STATE	localization, planning
object_detection	OBJECT_STATE	planning
localization	LOCATION	planning
planning	BEHAVIOUR_CMD	control
control	CONTROL_CMD	serial (IO out)

Topic	Message	Transport	Fields
CAMERA_FRAME	CameraFrame	SHM	width, height, stride, fmt, data_size
EGO_STATE	EgoState	SHM	yaw, pitch, roll, rpm
LANE_STATE	LaneState	SHM	heading_err_rad, offset_cm
CONTROL_CMD	ControlCmd	MQ	rpm, steer_deg
OBJECT_STATE	ObjectState	MQ	cls, distance, confidence
BEHAVIOUR_CMD	BehaviourCmd	MQ	target_speed, target_heading, mode
LOCATION	Location	MQ	x, y, heading

Table 32: Topic descriptors

2.3.2 IO

- Serial Bridge
- Web Server / Web Socket

2.3.3 Perception

The perception layer turns raw sensor streams (camera frames, serial telemetry) into structured world state that the planning layer can reason about. Each perception module is an independent node communicating over `libipc`.

2.3.3.1 Camera The camera module is the entry point of the visual pipeline. It acquires image frames and publishes them to a shared-memory topic so downstream vision nodes (lane detection, object detection) can read the latest frame without copying. During development the real camera is replaced by a video simulator (`camera_sim`) that decodes an MP4 file and emits frames at the source frame rate; the hardware capture node (`libcamera`) is planned but not yet implemented.

Node	Subscribe	Publish
<code>camera_sim</code>	— (reads MP4 file)	<code>CAMERA_FRAME</code> (SHM)

Table 33: Camera node communication

Algorithm theory. Video decoding extracts successive frames from an encoded stream; spatial resizing maps each frame onto a fixed grid using bilinear interpolation. The frame is then exposed through shared memory using a *latest-wins* policy: the publisher overwrites a fixed slot and the subscriber always reads the most recent frame, which is the correct semantic for high-rate sensor data where stale frames are useless.

Application in project. Each frame is resized to 320×180 BGR (3 channels, `stride` = 960 bytes, `fmt` = 0) and written to `CAMERA_FRAME` (SHM, depth = 3, drop policy NEW). The payload size is `sizeof(CameraFrame)` plus `stride` × `height` pixel bytes. The simulator loops the video on end-of-file and sleeps to match the source frame rate (default 30 fps).

Function	Description
<code>cv::VideoCapture::read</code>	Decode the next frame from the MP4 file
<code>cv::resize</code>	Resize the frame to 320×180
<code>ipc_publish_open</code>	Open the CAMERA_FRAME SHM publisher
<code>ipc_publish</code>	Write the frame (header + pixels) to SHM
<code>ipc_publish_close</code>	Release the publisher and unlink the SHM object

Table 34: Camera node functions

2.3.3.2 Lane Detection The lane-detection module receives a camera frame and estimates the vehicle’s position relative to the lane. It outputs two scalars: the *heading error* (angle between the lane tangent and the vehicle’s longitudinal axis) and the *lateral offset* (signed distance from the lane center). These feed the lateral controller for lane following.

Node	Subscribe	Publish
lane	CAMERA_FRAME (SHM)	LANE_STATE (SHM)

Table 35: Lane node communication

Algorithm theory. The pipeline combines several classical computer-vision techniques:

- **Inverse Perspective Mapping (IPM)** — a homography that warps the forward-facing camera view into a top-down (bird’s-eye) view, in which parallel lane lines remain parallel and curvature is measurable.
- **HLS color thresholding** — converting to the HLS color space and thresholding the Lightness (L) channel isolates bright lane markings from the dark road surface.
- **Histogram peak detection** — summing white pixels per column over the lower half of the image locates the left/right lane bases.
- **Sliding-window tracking** — starting from each base, a vertical stack of windows follows the lane upward, re-centering on the pixel mean.
- **Least-squares line fit** — the collected pixels are fit to a line $x = by + c$ via the normal-equation / SVD solver.
- **Exponential moving average (EMA)** — temporal smoothing reduces frame-to-frame jitter in the output.

Application in project. The node runs the following pipeline on every frame:

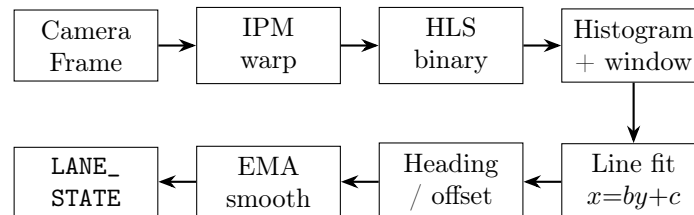


Figure 14: Lane detection pipeline

The IPM trapezoid, color threshold, and window parameters are tuned for the 320×180 frame:

Stage	Parameter	Value
IPM source (top)	TL_X, TR_X, TOP_Y	0.20, 0.80, 0.50
IPM source (bottom)	BL_X, BR_X, BOT_Y	0.00, 1.00, 0.80
Binary mask	L_MIN, L_MAX (HLS L)	200, 255
Sliding window	N_WINDOWS, MARGIN, MIN_PIX	9, 40 px, 30
Smoothing	ALPHA (EMA)	0.7
Scale	PIXEL_TO_CM (uncalibrated)	0.5

Table 36: Lane detection parameters

After fitting the lane line $x = by + c$, the heading error is the angle of the line with respect to the vertical axis, and the lateral offset is the horizontal distance between the lane reference and the image center (the vehicle body), scaled to centimeters:

$$\text{heading_err} = \arctan(b), \quad \text{offset_cm} = (x_{\text{lane}}|_{y=h-1} - \frac{w}{2}) \cdot \text{PIXEL_TO_CM}.$$

The right lane is preferred; when both lanes are detected the reference x is the midpoint of the two fitted lines, otherwise the node falls back to the left lane. The raw result is then filtered:

$$o_t = \alpha o_{t-1} + (1 - \alpha) o_t^{\text{raw}}, \quad \alpha = 0.7,$$

applied independently to heading error and lateral offset. On detection failure the previous smoothed value is held.

Function	Description
<code>ipm_matrix(w,h)</code>	Build the perspective transform from the source trapezoid
<code>ipm(src,M)</code>	Warp the frame to a bird's-eye view
<code>binary_mask(bgr)</code>	HLS L-channel threshold $\rightarrow$ binary image
<code>ipm_src_points(w,h)</code>	Return the 4 trapezoid points (debug overlay)
<code>detect(bw)</code>	Histogram + sliding window + line fit $\rightarrow$ <code>DetectResult</code>
<code>Tracker::smooth(cur)</code>	EMA filter across consecutive frames

Table 37: Lane detection functions

2.3.3.3 Object Detection *Planned:* run a YOLOv8n model (ONNX) on `CAMERA_FRAME` and publish detected object class, distance and confidence to the `OBJECT_STATE` topic. Not yet implemented.

2.3.3.4 Ego State The ego-state module is the bridge between the STM32 and the perception layer. The serial node reads the UART telemetry frames (`@IMU` and `@RPM`), parses them, and publishes the raw values directly on `EGO_STATE`. The message carries the IMU Euler angles and the motor RPM as received; consumers (localization, control) derive heading and speed from these raw values themselves, so no separate processing node is needed.

Node	Subscribe	Publish
serial	— (reads UART)	EGO_STATE (SHM)

Table 38: Serial / ego-state node communication

Algorithm theory. Linear speed is derived from the motor angular velocity through the drivetrain: dividing motor RPM by the gear ratio yields wheel RPM, and multiplying by the wheel circumference converts to linear velocity. The heading is taken directly from the on-chip fused yaw of the IMU. These conversions are performed by the consumer, not by the serial node.

Application in project. With a gear ratio of 7:1 and wheel radius 6.5 cm, a consumer recovers:

$$\text{heading} = \text{yaw}, \quad \text{speed} = \frac{\text{rpm}}{7} \cdot 2\pi \cdot \frac{6.5}{60} \text{ [cm/s]}.$$

EgoState carries yaw, pitch, roll (degrees) and rpm (motor RPM) as published by the serial node.

2.3.4 Localization

Planned: dead reckoning from EGO_STATE (integrating speed and heading over time) to produce a 2D pose published on the LOCATION topic. Not yet implemented.

2.3.5 Planning

- Input
- Output
- Algorithm

2.3.6 Control

- Input
- Output
- Algorithm

2.4 Monitor

- Web Server
- Dashboard

2.5 Simulation

- Visualization
- Direction Simulation

3 Source & Description